DEVELOPING A LIGHTWEIGHT AND SECURE OTA DELIVERY MECHANISM

Project ID : 25-26j-058
Individual Final Report Thesis

IT22365996
Mandira U.L.L.K

BSc. (Hons) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology Sri Lanka

April 2026

DEVELOPING A LIGHTWEIGHT AND SECURE OTA DELIVERY MECHANISM

Project ID : 25-26j-058

IT22365996
Mandira U.L.L.K

Supervisor : Mr. Kavinga Yapa Abewardena
Co supervisor : Ms. Dinithi Pandithage

BSc. (Hons) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology Sri Lanka

April 2026

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature	Date
Mandira U.L.L.K	IT22365996		25.04.2026

Name of the supervisor: Mr. Kavinga Yapa Abeywardena Name of co-supervisor: Ms. Dinithi Pandithage
The above candidate has carried out research for the bachelor's degree dissertation under my supervisor.

Signature: ……
Supervisor: Mr. Kavinga Yapa Abeywardena

Signature: ……
Co supervisor: Ms. Dinithi Pandithage

Data: 25.04.2026

ABSTRACT

OTA firmware update techniques that aim at resource-constrained embedded devices are subject to contradicting requirements in terms of minimality of payload size, cryptographic verification, resilience to rebooting in the middle of flash operations and scalability in their distribution in the constrained environment of platforms like the ESP32. In this regard, the existing strategies either send complete firmware files irrespective of the update requirements or employ binary patching for any case of firmware differences.

The current study aims to offer the design and implementation of the OTA delivery component of the SecureTrustOTA project for the ESP32 platform. Specifically, three major innovations are proposed by the paper. The first one involves the use of a Decision Engine for comparing the two firmware instances in question based on four criteria, namely Structural Change Ratio (SCR), Fragmentation Score (FS), Reconstruction Complexity (RC) and Instability Risk (IR). Then, based on the calculated weighted Decision Score (DS), the optimal solution for the transfer is identified as transferring the complete firmware image, standard delta or instruction-aware delta. Secondly, a process for the generation of the instruction-aware delta is proposed which utilizes the Xtensa ISA for normalization of control flow operands. Third, a Peer-Assisted Mesh Distribution layer leverages ESP-NOW to enable a seed device to rebroadcast an encrypted update to co-located peers, reducing server fan-out by an order of magnitude in local deployments.

Security is maintained end-to-end through REM 1.1, a custom streaming encryption protocol combining AES-256-CFB, HMAC-SHA256, and HKDF-SHA256, encapsulating a self-describing OTA-PKG v1 envelope that allows the device to parse and apply any payload type after a single decryption pass. Backward compatibility with REM 1.0 devices is preserved throughout.

Keywords: *Over-the-air update, delta patching, instruction-aware diffing, decision engine, peer-assisted distribution*

ACKNOWLEDGMENT

I would like to sincerely express my gratitude to supervisors, Mr. Kavinga Yapa Abeywardena and Ms. Dinithi Pandithage, Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology, for their constant support and guidance regarding the development of this research proposal. Their knowledge and constructive feedback have been invaluable in shaping the direction of this project and in addressing possible challenges.

I would also like to thank to all those who have in one way or the other participated in the accomplishment of this study. Lastly, my appreciation goes to the coordinators and the design and analysis lecturer of this course for all round support during the whole proposal phase. Their guidance has been fundamental in laying a strong foundation for this project.

TABLE OF CONTENTS

Contents

DECLARATION III

ABSTRACT IV

ACKNOWLEDGMENT V

TABLE OF CONTENTS VI

LIST OF FIGURES VIII

LIST OF TABLES VIII

LIST OF ABBREVIATIONS IX

CHAPTER 1: INTRODUCTION X

1.1 RESEARCH BACKGROUND X
1.2 RESEARCH SCOPE XI
1.3 BACKGROUND AND LITERATURE SURVEY XII
1.4 RESEARCH GAP XV
1.5 RESEARCH AREA XVII
1.6 SIGNIFICANCE OF THE RESEARCH XVIII
1.7 RESEARCH PROBLEM XX
1.8 RESEARCH QUESTIONS XX
1.9 RESEARCH OBJECTIVES XXI

CHAPTER 2: RESEARCH METHODOLOGY XXIII

2.1 OVERVIEW OF METHODOLOGY XXIII
2.3 SYSTEM DIAGRAM XXIII
2.3 COMPONENT DIAGRAM XXV
2.4 SEVER PATH AND DEVICE PATH XXVII
2.5 DEVICE SIDE APPLIER XXVIII
2.6 IMPLEMENTATION TOOLS AND TECHNOLOGY SELECTION XXIX
Server Side (FastAPI Backend) xxix
Frontend (React Dashboard) xxix
Device Side (ESP32 Firmware) xxix
2.7 DEVELOPMENT FLOW XXX
2.8 EVALUATION PLAN XXXI
2.9 COMMERCIALIZATION ASPECTS OF THE PRODUCT XXXII
2.10 TESTING AND IMPLEMENTATION XXXIII
Test Cases xxxiii

CHAPTER 3: PROJECT REQUIREMENTS XXXVI

3.1 FUNCTIONAL REQUIREMENTS XXXVI
3.2 NON FUNCTIONAL REQUIREMENTS XXXVI
3.3 USER REQUIREMENTS XXXVII
3.4 SYSTEM REQUIREMENTS XXXVII

CHAPTER 4: RESULTS AND DISCUSSION XXXVIII

4.1 RESULTS XXXVIII
Decision Engine Performance xxxviii

Instruction Aware Normalization xxxix
Envelope Overhead xxxix
Peer Assisted Mesh Distribution xxxix
System Integration xl
4.2 Research Findings XL
4.3 Discussion XLI
CHAPTER 5: CONCLUSION XLIV
REFERENCE XLV
APPENDIX A: DECISION ENGINE XLVI
A.1 Decision Score Formula XLVI
A.2 Analyze Changes XLVI
APPENDIX B: REM 1.1 ENCRYPTION XLVII
B.1 Key Derivation (HKDF-SHA256) XLVII
B.2 REM Encrypt and Decrypt XLVII
APPENDIX C: INSTRUCTION AWARE DELTA XLVIII
C.1 Normalize Buffer and Build Address Map XLVIII

LIST OF FIGURES

Figure 1: System diagram ……25
Figure 2: Component diagram ……..27
Figure 3: Server Path and Device Path……..29
Figure 4 : Device Side Applier ……30

LIST OF TABLES

Table 1: List of Abbreviations ……..……9
Table 2: Research gap ……..……16
Table 3: Memory profile ……..……31
Table 4: Test case 1 ……..……33
Table 5: Test case 2……..……34
Table 6: Test case 3 ……..……34
Table 7: Test case 4 ……..……35
Table 8: Decision engine performance ……38
Table 9: Envelope overhead ……38
Table 10: peer assisted mesh distribution ……...……..……39

LIST OF ABBREVIATIONS

Abbreviation	Definition
OTA	Over The Air
IoT	Internet of Things
ECC	Elliptic Curve Cryptography
AES - CCM	Advanced Encryption Standard Counter with CBC MAC
IR	Instability Risk
SCR	Structural Change Ratio
FS	Fragmentation Score
RC	Reconstruction Complexity
OTA PKG v1	Over the Air Package version 1
DS	Decision Score
mTLS	Mutual Transport Layer Security
ECDSA	Elliptic Curve Digital Signature Algorithm
TLS	Transport Layer Security

Table1: List of Abbreviations

CHAPTER 1: INTRODUCTION

1.1 Research Background

Over the air firmware update mechanisms have become a fundamental requirement in modern IoT deployments. Since the devices are not readily accessible once deployed, wireless delivery of software patches is vital in order to maintain functionality, plug security gaps, and increase the operational lifespan [1]. ESP32, which features a dual core architecture, 4 MB internal flash memory, and out of the box dual OTA partition support using ESP IDF, can be used as a basis for the design of OTA update mechanisms [2]. However, although the ESP IDF OTA solution supports wireless updates, it does not have the functionality for delta patch generation, adaptive payloads, and even lacks any cryptographic measures apart from HTTPS transport [2]. These concerns that must be addressed entirely at the application layer. This gap motivates the development of a more capable and security conscious OTA delivery pipeline tailored to the constraints of the platform.

Binary delta patching decreases the size of an OTA payload, transferring only the difference between two firmware versions as opposed to a full payload. The bsdiff utility for efficient byte-wise diff generation is frequently used for generating binary deltas. [3]. For IoT devices operating over bandwidth constrained wireless links, the reduction in transmitted bytes directly translates to faster updates, lower energy consumption, and reduced risk of corruption during transfer [4]. However, bsdiff operates purely at the byte level with no understanding of the underlying binary structure. When firmware changes are extensive such as when a new module is introduced or global data layouts shift, the resulting patch can approach the size of the full binary, eliminating any benefit [4]. This fundamental limitation of unconditional delta patching is a key problem that an adaptive decision layer must address.

A significant source of patch inflation in compiled firmware arises not from logical code changes but from address relocations. In architectures such as Xtensa, used by the ESP32, instructions like L32R, CALL8, CALL12, and J encode PC relative offsets in their operand bytes. When a function shifts in memory due to an unrelated addition elsewhere in the binary, every one of these operand fields changes even though no logic has been modified. Standard byte level diffing treats these operand changes as genuine differences, producing unnecessarily large and complex patches. Instruction aware diffing addresses this by normalizing the operand bytes of known control flow instructions to a canonical value before the diff is computed, and separately recording the original operand values for replay during patch application. This approach makes the diff sensitive only to true logical changes rather than layout artefacts, resulting in smaller and more stable patches for firmware that has undergone primarily structural reorganization.

In order to achieve strong cryptographic security of over-the-air update mechanisms while working within constraints imposed by limited memory and computational power available on a device, confidentiality can be achieved via AES encryption, whereas integrity and authenticity can be ensured using either MAC or digital signature schemes. The Encrypt then MAC construction, where the HMAC is computed over the ciphertext rather than the plaintext, is cryptographically preferred because it ensures the integrity of the encrypted payload before any decryption work is performed. Asymmetric AEAD schemes such as AES-GCM are theoretically elegant but impose a practical constraint. The authentication tag cannot be verified until the entire ciphertext has been received, requiring full in memory buffering. On a device with limited RAM handling firmware sized payloads, this is impractical. A streaming Encrypt then MAC approach, where decryption and HMAC verification proceed in small chunks, is far better suited to embedded contexts.

Delivering firmware updates from a central server to a large number of co located devices creates a bandwidth cost that scales linearly with device count. In scenarios where many devices share the same physical space, this fan out is both inefficient and unnecessary. Peer assisted distribution addresses this by designating one device as a local relay that downloads the update once and rebroadcasts it to neighboring peers [8]. ESP-NOW, a connectionless, infrastructure free communication protocol native to the ESP32, is well suited to this role due to its low latency and independence from a wireless access point. However, enabling a device to relay firmware over ESP NOW without weakening the security guarantees of the update pipeline requires careful layering. Per packet integrity checks to prevent garbage assembly, combined with full end to end cryptographic verification of the reassembled payload before any flashing occurs [9].

1.2 Research Scope

This research focuses on the design and implementation of a lightweight, secure, and adaptive OTA firmware delivery mechanism for the ESP32 microcontroller as an individual component of the broader SecureTrustOTA system. The scope is bounded to the server side update pipeline, the cryptographic packaging layer, and the on device firmware application logic that covering the full path from firmware ingestion on the backend to successful partition flashing on the device.

The work specifically addresses three areas of improvement over the baseline SecureTrustOTA v1 implementation. The first is the introduction of an adaptive decision engine that evaluates each firmware pair using four quantitative metrics which are Structural Change Ratio, Fragmentation Score, Reconstruction Complexity, and Instability Risk and selects the most appropriate delivery strategy automatically. This eliminates the static assumption that delta patching is always beneficial and replaces it with an evidence based selection between full image transfer, standard delta, and instruction aware delta [4]. The decision engine is configurable through the FastAPI backend and its weights and threshold are tunable per deployment.
The second area covers instruction aware delta generation, which extends standard byte level

binary diffing to account for the Xtensa instruction set architecture of the ESP32. By identifying and normalizing control flow operands in both the old and new firmware prior to diffing, the pipeline produces patches that reflect true logical changes rather than address relocation artefacts [5]. The resulting IADELTA1 container format carries both the normalized patch and the operand maps required for faithful reconstruction on the device.

The third aspect relates to distribution scalability by means of creating a peer assisted mesh layer using ESP-NOW. The current scope includes the seed and leaf role designation, the verification of each packet with HMAC in the broadcast, reassembling of order chunks at leaf nodes, as well as falling back to HTTP downloads in case of mesh timeout [8]. The mesh layer works solely on the basis of the existing cryptographic stack and does not compromise any of its guarantees.

The cryptographic layer which is REM 1.1 and the self describing OTA-PKG v1 envelope fall within scope as the packaging and security substrate that all three delivery strategies depend upon. Backward compatibility with REM 1.0 devices is maintained throughout. The React dashboard and FastAPI backend are within scope insofar as they expose the decision engine configuration, firmware upload workflow, and update queuing but their internal design is not the primary focus of this component.

Out of scope for this component are the key provisioning and device registration mechanisms, the broader SecureTrustOTA group authentication system, cloud deployment infrastructure.

1.3 Background and Literature Survey

Over the Air Firmware Updates in IoT Systems

Over the air firmware updates have become a foundational requirement in modern IoT deployments. As devices are frequently installed in physically inaccessible locations, remote update mechanisms are essential for delivering security patches, configuration changes, and new functionality throughout a device's operational lifetime [1]. The IETF Software Updates for Internet of Things (SUIT) Working Group identified this need and published RFC 9019, which defines a firmware update architecture and provides the motivation for a standardized, transport agnostic manifest format for describing and protecting firmware updates on resource constrained devices [1].

The ESP32 microcontroller, with its dual core processor, integrated Wi-Fi and Bluetooth radios, and native dual OTA partition support via the ESP-IDF framework, is one of the most widely deployed platforms for IoT firmware update research [2]. While ESP-IDF's native OTA library supports Wi-Fi based updates, it lacks multiprotocol coordination and version aware delivery mechanisms, leaving security, delta patching, and adaptive distribution to be addressed at the

application layer [2]. This gap motivates the design of a more comprehensive update pipeline that addresses payload efficiency, cryptographic security, and scalable distribution simultaneously.

Binary Delta Patching

Binary delta patching reduces the size of firmware updates by transmitting only the differences between two versions rather than the complete image. The bsdiff algorithm, developed by Colin Percival, uses suffix sorting to exploit the statistical structure of executable files and routinely produces patches significantly smaller than those generated by other binary differencing tools [3]. This makes it particularly attractive for IoT firmware update pipelines where bandwidth is constrained and update reliability is critical [4].

However, bsdiff faces well known issues in its application to compiled firmware. A major challenge is the pointer problem which is a small source level adjustments shift the relative positions of code and data blocks, causing all pointers that cross the modified region to become outdated and appear as changes in the diff, even though no logic has changed [4]. A significant divergence in the firmware, as seen when a completely new subsystem is implemented or the memory layout is drastically altered results in a patch size nearly identical to the binary itself, rendering the use of delta patching meaningless in terms of reducing network bandwidth requirements [3]. This limitation motivates an adaptive decision layer capable of determining when a delta is truly beneficial and when a full image transfer is preferable.

Instruction Aware and Architecture Sensitive Diffing

A well established approach to improving patch quality for compiled binaries is to preprocess the input before diffing, normalizing architecture specific fields that change due to relocation rather than logic. Google's Courgette, which powers Chrome browser updates, is essentially bsdiff with a preprocessing step that disassembles the executable to normalize relative addresses before running the core algorithm, then the same fundamental idea applied at the binary level. This normalization ensures that address driven changes do not inflate the patch unnecessarily [6].

The Xtensa ISA, used by the ESP32, presents a specific class of this problem. Instructions such as L32R, CALL8, CALL12, and J encode PCrelative offsets directly in their operand bytes. Any recompilation that shifts a function or literal pool causes these operand fields to change throughout the binary, even when the actual program logic is identical. Algorithms such as Courgette and its successor Zucchini demonstrate that disassembling the executable to an intermediate normalized form, diffing the normalized representation, and then reassembling on the client can reduce patch sizes by an order of magnitude compared to raw bsdiff [5]. The instruction aware delta pipeline implemented in this work applies the same principle, targeting the specific contraflow instruction types of the Xtensa ISA.

Cryptographic Security for Constrained OTA Pipelines

Securing OTA updates on resource constrained devices requires careful selection of cryptographic primitives that provide strong guarantees without exceeding the device's RAM or processing budget. RFC 9019 identifies integrity, authenticity, and confidentiality as the three core security properties that a firmware update mechanism must provide, and highlights that constrained devices require solutions that are compatible with their limited hardware capabilities [1].

AES is the standard choice for confidentiality in embedded systems due to its hardware acceleration support on platforms including the ESP32 [6]. AES supports multiple key lengths are 128-bit, 192-bit, and 256-bit allowing deployments to balance security strength against processing overhead according to their requirements. For integrity and authenticity, HMAC-SHA256 combined with HKDF-based key derivation provides a well understood and efficient construction [10] [11]. The Encrypt then MAC ordering where the HMAC is computed over the ciphertext is cryptographically preferred over alternatives because it ensures the integrity of the encrypted payload is verified before any decryption work is performed [7]. This ordering is especially important in streaming contexts, where an attacker could otherwise exploit the decryption process before the authentication tag is checked. AEAD schemes such as AES-GCM are theoretically more compact but require full ciphertext buffering before tag verification, which is impractical on a device with limited RAM processing multi kilo byte firmware payloads [12].

Peer Assisted and Mesh Based Firmware Distribution

Distributing firmware from a central server to a large group of collocated IoT devices creates a server bandwidth cost that scales linearly with device count. Research into mesh based and peer assisted distribution addresses this by enabling devices to relay updates locally [8]. Studies on largescale IoT mesh firmware rollouts demonstrate that the quality of service of the shared wireless medium including throughput and join time are critical performance indicators, and that concurrent transmission strategies are necessary to maintain reliability as network size grows [8].

Prior work has explored OTA firmware update systems built on lightweight mesh network protocols, combining low power mesh routing with over the air delivery to reduce infrastructure dependence in smart urban deployments [9]. ESP-NOW, Espressif's connectionless peer to peer protocol native to the ESP32, offers a practical implementation substrate for local mesh distribution: it operates without a wireless access point, supports broadcast and unicast modes, and imposes low per packet overhead. Its 250-byte payload limit requires firmware to be fragmented into chunks for relay, making per packet integrity verification an important design consideration. The peer assisted mesh layer implemented in this work builds on these foundations, adding HMAC-protected chunked broadcast and automatic server fallback on timeout, while preserving full end to end REM and ECDSA verification on the reassembled payload [13].

1.4 Research Gap

Existing OTA firmware update solutions for constrained IoT devices address individual aspects of the update problem but fall short when evaluated against the combined requirements of payload efficiency, cryptographic security, self description, and scalable distribution. The ESP-IDF native OTA mechanism provides a reliable dualpartition flashing infrastructure but offers no delta patching, no payload authentication beyond HTTPS transport, and no mechanism for the device to identify the type of payload it has received. Tools such as bsdiff provide strong binary differencing but operate without any awareness of the underlying instruction set architecture, producing inflated patches whenever firmware layouts shift due to recompilation. Cryptographic frameworks such as AES-GCM and ChaCha20-Poly1305, while theoretically sound, require full ciphertext buffering before authentication tag verification, a constraint that is incompatible with the streaming decryption model required on RAM limited devices. Distribution architectures based purely on server to device HTTP delivery scale linearly with device count and provide no mechanism for local relay, creating unnecessary bandwidth overhead in co located deployments.
No existing open solution for the ESP32 integrates all of these concerns into a single, cohesive pipeline. This baseline addressed encryption and delta patching in combination but treated delta patching as unconditional, lacked payload self description, and offered no peer assisted distribution. The three contributions of this component, the decision engine, instruction aware delta generation, and ESP-NOW mesh layer, directly address these identified gaps.

- Absence of adaptive payload selection: No existing lightweight OTA solution for the ESP32 incorporates a data driven mechanism to determine whether delta patching is beneficial for a given firmware pair. Existing tools apply binary differencing unconditionally, meaning heavily diverged firmware pairs incur reconstruction overhead with no bandwidth saving. A quantitative decision layer evaluating structural change, fragmentation, reconstruction cost, and control flow risk is absent from current open solutions.
- Instruction set blindness in binary delta generation: Existing differencing tools including bsdiff operate purely at the byte level with no knowledge of the target architecture. For Xtensa-compiled ESP32 firmware, PCr elative operand fields in instructions such as L32R, CALL8, and J change with every recompilation even when logic is unchanged, causing unnecessary patch inflation. No published OTA solution addresses this layout riven bloat through ISA aware normalization.
- Lack of self describing payload format: Existing ESP32 OTA pipelines rely on implicit assumptions or external metadata to determine how a received payload should be applied. No standardized mechanism embedded within the encrypted payload itself allows the device to identify and dispatch between full image, delta, and instruction aware update types after decryption preventing safe coexistence of multiple payload formats within the same deployment.

Limitation in Existing Systems	Proposed Solution
Delta patching applied unconditionally regardless of firmware divergence	Decision Engine computes DS from SCR, FS, RC, IR and selects full, delta, or IA strategy per firmware pair
Standard bsdiff is blind to ISA level address relocations, inflating patches	Instruction Aware Delta normalizes Xtensa control low operands before diffing, reducing patch size for layout riven changes
No self describing payload: device cannot determine update type post decryption	OTA-PKG v1 envelope carried inside REM 1.1 plaintext identifies payload type via flags after a single decryption pass
AEAD schemes require full ciphertext buffering before authentication	REM 1.1 uses streaming Encrypt then MAC (AES-256-CFB + HMAC-SHA256) in 1 KB chunks, no full file RAM buffer needed
Server to device delivery scales linearly with device count	Peer Assisted Mesh via ESP-NOW allows one seed to rebroadcast to N-1 leaves, reducing server fan-out proportionally
No backward compatibility between update protocol versions	REM 1.1 decryptor accepts both R1.1 and legacy REM1 magic bytes, older devices continue to receive updates
Rogue server and malicious peer attacks not jointly mitigated	ECDSA P-256 signature verified over the full encrypted payload on every device regardless of delivery path

Table2: Research gap

1.5 Research Area

Embedded Systems and IoT Firmware Management

This research operates within the domain of embedded systems engineering, specifically targeting firmware lifecycle management on resource constrained microcontrollers. The ESP32 platform with its 4 MB flash, dual OTA partitions, and Xtensa LX6 processor that represents a widely deployed class of IoT device where update efficiency and reliability are critical operational concerns.

Binary Delta Patching and Adaptive Payload Selection

A core research area is the application of binary differencing techniques to firmware update pipelines. This includes the study of bsdiff as a foundational algorithm, its known limitations under high firmware divergence, and the design of decision layer heuristics that determine when delta patching yields a net benefit over full image transfer. The decision engine developed in this work computing a weighted score across four structural and risk metrics falls within this area.

Architecture Aware Binary Analysis

This research engages with low level binary analysis of Xtensa ISA compiled firmware. Identifying control flow instructions, normalizing PC-relative operands, and constructing operand maps for faithful reconstruction on device are tasks that require understanding of both the instruction set architecture and the ELF binary format. This area intersects compiler output analysis, binary instrumentation, and patch generation.

Applied Cryptography for Constrained Devices

The design and implementation of REM 1.1 falls within applied cryptography for embedded systems. This covers symmetric encryption (AES-256-CFB), keyed hashing (HMAC-SHA256), key derivation (HKDF-SHA256), and asymmetric signature verification (ECDSA P-256) all implemented within the memory and processing constraints of the ESP32 using mbed TLS primitives already present in the platform's ROM.

Distributed and Peer Assisted Communication in IoT Networks

The peera ssisted mesh layer places this research within the area of distributed IoT communication. Specifically, it addresses local firmware relay over ESP-NOW, per packet integrity verification, chunk based reassembly, and graceful fallback strategies all within a security model that preserves end to end cryptographic guarantees regardless of how the payload was delivered.

1.6 Significance of the Research

Firmware update mechanisms for constrained IoT devices have historically been designed around one of two extremes which are transmitting full firmware images unconditionally, or applying binary delta patches regardless of whether doing so is beneficial. Both approaches carry real costs. Full image transfers consume unnecessary bandwidth when changes are small. Unconditional delta patching introduces complexity and on device risk when firmware has diverged significantly producing patches that are nearly as large as the binary they replace while still requiring the reconstruction step. Neither approach adapts to the actual nature of the change being deployed.
The significance of this research lies in replacing that static assumption with an evidence based, adaptive delivery pipeline. By computing a Decision Score from four quantitative metrics that each capture a distinct dimension of firmware divergence structural spread, spatial fragmentation, reconstruction cost, and control flow risk. The decision engine selects the delivery strategy most appropriate for each specific firmware pair. This means devices receive smaller payloads on average, spend less time in the vulnerable mid flash state, and are exposed to lower reconstruction risk than with any single fixed strategy.

The instruction aware delta pipeline extends this contribution further. By normalizing Xtensa control flow operands before diffing, the system produces patches that reflect genuine logical changes rather than address relocation artefacts. For the large class of firmware updates that involve primarily adding a function, adjusting a constant, or reorganizing a module, changes that cause widespread address shifts without altering program logic, the instruction aware approach can yield substantially smaller patches than standard bsdiff. This directly reduces over the air transmission time, flash write cycles, and the window of vulnerability during which a device is partially updated.

The REM 1.1 protocol and OTA-PKG v1 envelope contribute a self describing, streaming compatible cryptographic packaging layer that closes a gap present in the original SecureTrustOTA v1 design. Previously, the device had no way to determine from the decrypted payload alone what type of update it had received, it assumed a bsdiff patch. The envelope resolves this ambiguity after a single decryption pass, enabling backward compatible support for three distinct payload types without relying on URL conventions or out of band metadata. The streaming Encrypt then MAC construction ensures that devices with limited RAM can verify and apply arbitrarily large updates without ever buffering the full payload in memory.

Finally, the peer assisted mesh layer addresses the scalability dimension of firmware distribution. In deployments where multiple ESP32 devices share a physical space, the ability for one seed device to rebroadcast an encrypted update to its neighbours over ESP-NOW reduces server bandwidth consumption proportionally to the number of leaves in the group. This benefit is achieved without relaxing any security guarantee. The same REM tag and ECDSA signature

verification runs on every device regardless of whether the payload arrived from the server or a peer. For resource constrained deployments where network bandwidth or server capacity is limited, this represents a meaningful reduction in operational cost.

Taken together, these contributions advance the state of practice in secure, efficient OTA firmware delivery for constrained embedded platforms combining adaptive intelligence at the payload selection layer, architectural awareness at the patch generation layer, cryptographic rigour at the packaging layer, and distribution scalability at the network layer.

1.7 Research Problem

The baseline SecureTrustOTA system demonstrated that combining bsdiff delta patching with AES-256-CFB encryption is a viable approach to secure OTA delivery on the ESP32. However, three fundamental problems remained unresolved that limited its practical utility in real world deployments.

The first problem is the absence of adaptive payload selection. Version 1 emitted a bsdiff patch for every firmware pair regardless of how different the two binaries were. When firmware diverges significantly due to the addition of a new module, a change in memory layout, or widespread symbol relocation. The resulting patch can approach the size of the full binary. In this scenario, delta patching adds reconstruction overhead and on device risk without delivering any bandwidth saving. There was no mechanism to detect this condition and fall back to full image transfer, nor any way to exploit ELF level structure to produce a smaller patch when it was available.

The second problem is the absence of a self describing payload format. After decryption, the device had no way to determine from the byte stream alone what type of update it had received. The system implicitly assumed that all decrypted payloads were bsdiff patches an assumption that breaks the moment multiple payload types must coexist. Extending the system to support full images, standard deltas, and instruction aware deltas requires a reliable, version safe dispatch mechanism that operates entirely within the decrypted payload itself.

The third problem is distribution scalability. In deployments where multiple ESP32 devices share a physical space, delivering the same encrypted update from the server to every device individually is wasteful. Server bandwidth consumption scales linearly with device count, and there is no mechanism for devices to cooperate in distributing an update locally once one device has already downloaded it.

These three problems collectively limit the system's efficiency, extensibility, and scalability and they are the precise problems that this component is designed to solve.

1.8 Research Questions

1. Under what measurable conditions does binary delta patching yield a smaller and safer payload than full firmware transfer, and can those conditions be quantified into a reliable decision score?
2. To what extent does normalizing Xtensa ISA control flow operands prior to binary diffing reduce patch size for firmware updates driven primarily by address relocation rather than logic change?
3. Can a streaming Encrypt then MAC protocol combining AES-256-CFB and HMAC-SHA256 provide equivalent end to end security to AEAD schemes while remaining compatible with the RAM constraints of the ESP32?
4. How can a self describing payload envelope be designed to allow an ESP32 device to

correctly identify and dispatch full, delta, and instruction aware update payloads after a single decryption pass, while maintaining backward compatibility with legacy devices?

5. To what degree does a peer assisted ESP-NOW mesh distribution layer reduce server bandwidth consumption in co located ESP32 deployments, and what security guarantees can be preserved when the delivery path passes through an untrusted peer?

1.9 Research Objectives

The primary objective of this research is to design, implement, and validate a lightweight, secure, and adaptive over the air firmware delivery mechanism for the ESP32 microcontroller that addresses the key limitations of unconditional delta patching, opaque payload formats, and server only distribution present in the SecureTrustOTA baseline.

To achieve this primary objective, the following specific research objectives are defined:

Design an adaptive firmware payload decision engine To design and implement a quantitative decision engine that evaluates each firmware pair using four measurable metrics which are Structural Change Ratio, Fragmentation Score, Reconstruction Complexity, and Instability Risk and computes a weighted Decision Score to automatically select the most appropriate update strategy (full image, standard delta, or instruction aware delta) for each specific firmware pair, eliminating the static assumption that delta patching is always beneficial.

Develop an instruction aware delta generation pipeline To develop a binary delta generation pipeline that understands the Xtensa instruction set architecture of the ESP32, identifies control flow instruction sites including L32R, CALL8, CALL12, and J, normalizes their PC relative operand bytes before diffing, and packages the resulting patch in a self contained IADELTA1 container with operand maps sufficient for faithful reconstruction on the device producing significantly smaller patches for firmware pairs where address relocation is the primary source of binary divergence.

Design and implement the REM 1.1 streaming encryption protocol To design and implement REM 1.1, a custom streaming encryption protocol combining AES-256-CFB confidentiality, HMAC-SHA256 integrity under the Encrypt then MAC construction, and HKDF-SHA256 key derivation, capable of encrypting and decrypting firmware sized payloads in 1 KB chunks on the ESP32 without requiring full file RAM buffering, while maintaining backward compatibility with legacy REM 1.0 devices.

Define a self describing OTA payload envelope To define and implement the OTA-PKG v1 envelope format, carried inside the REM 1.1 plaintext, that embeds the update strategy flag, firmware version strings, and SHA-256 digest of the new binary enabling the ESP32 device to unambiguously identify the payload type and dispatch to the correct apply handler after a single decryption pass, without relying on external metadata or URL conventions.

Implement a peer assisted ESP-NOW mesh distribution layer To implement a peer assisted firmware distribution layer over ESP-NOW that enables a designated seed device to download an encrypted update once from the server and rebroadcast it to co located leaf devices in HMAC-protected 200byte chunks, with automatic fallback to direct HTTP on 60second mesh timeout reducing server bandwidth consumption from O(N) to O(1) relative to device count while preserving all end-to-end cryptographic guarantees.

Validate end to end integration on physical ESP32 hardware To integrate all three contributions through the FastAPI backend and React dashboard and validate the complete pipeline on physical ESP32 hardware, confirming that ECDSA signature verification, REM streaming decryption, HMAC tag verification, SHA-256 digest validation, and OTA partition flashing all complete correctly across all supported payload types and delivery paths.

CHAPTER 2: RESEARCH METHODOLOGY

2.1 Overview of Methodology

This research follows an iterative, agile development methodology in which the system was designed, implemented, and validated in successive cycles. Each iteration targeted one of the three principal contributions, the decision engine, instruction aware delta pipeline, and peer assisted mesh layer, building incrementally on a shared cryptographic substrate. This approach allowed each component to be tested and refined independently before being integrated end to end through the FastAPI backend, React dashboard, and ESP32 firmware.

2.3 System Diagram

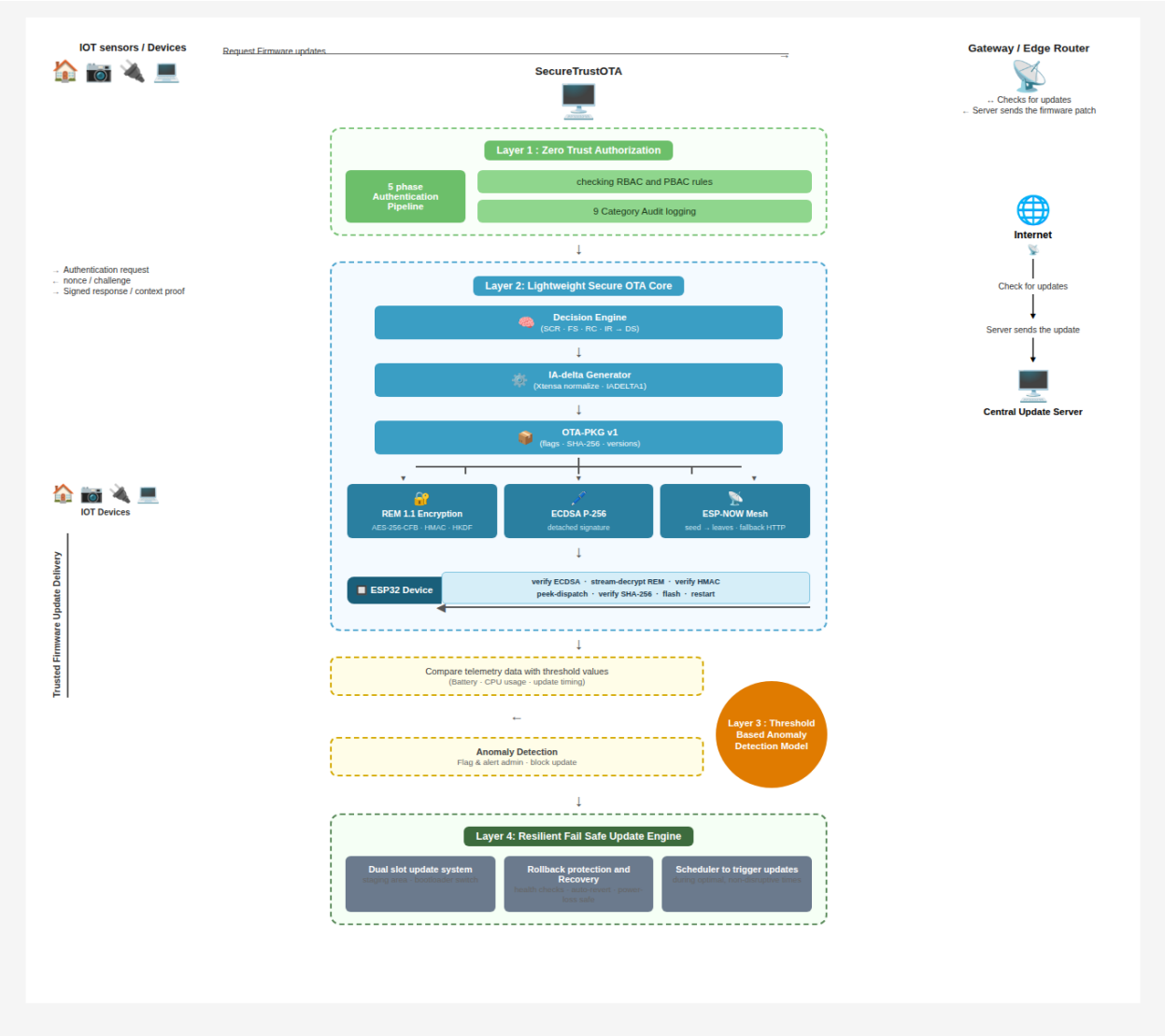


Figure 1: System diagram

Layer 1 Zero Trust Authorization

The first layer enforces strict access control before any update operation is initiated. A five phase authentication pipeline is used to validate devices through challenge response mechanisms and signed context verification. Role based and policy based access control rules are applied to ensure that only authorized entities can request or receive firmware updates. In addition, audit logging is performed across multiple categories to maintain traceability and accountability throughout the update process.

Layer 2 Lightweight Secure OTA Core

This layer represents the core contribution of the system and is responsible for intelligent update generation, secure packaging, and efficient delivery.

At the beginning, the Decision Engine evaluates firmware differences using Structural Change Ratio, Fragmentation Score, Reconstruction Complexity, and Instability Risk. Based on a computed decision score, the system selects the most suitable update strategy, which can be a full image, standard delta, or instruction aware delta.

The Instruction Aware Delta Generator processes firmware using Xtensa specific normalization to eliminate non logical differences such as address shifts. This significantly reduces patch size and improves efficiency.

The selected payload is then encapsulated into the OTA PKG v1 format, which acts as a self describing container including metadata, versioning, and payload type information.

For security, the system applies REM 1.1 encryption using AES 256 CFB, HMAC, and HKDF to ensure confidentiality and integrity. In parallel, ECDSA based digital signatures provide authenticity verification.

For scalable distribution, the system integrates ESP NOW based mesh communication, where a seed device rebroadcasts updates to nearby devices, reducing server load.

On the device side, the ESP32 performs signature verification, streaming decryption, HMAC validation, payload interpretation, and finally flashes the firmware to the appropriate partition before restarting.

Layer 3 Threshold Based Anomaly Detection

This layer monitors device telemetry during the update process to detect abnormal behavior. Parameters such as battery level, CPU usage, and update timing are continuously evaluated against predefined threshold values.

If deviations are detected, the system triggers anomaly alerts, flags the device, and can block the update process to prevent potential failures or malicious activity. This ensures that only safe and stable updates are applied.

Layer 4 Resilient Fail Safe Update Engine

The final layer ensures reliability and recovery in case of failures during the update process. A dual slot update mechanism is used, where the new firmware is written to an inactive partition while the current version remains intact. This allows safe switching between firmware versions. Rollback protection and recovery mechanisms are implemented to restore the previous firmware if the update fails or becomes unstable. Additionally, a scheduling component enables updates to be triggered during optimal time windows to minimize disruption.

Overall, the system begins with secure authentication, followed by intelligent update selection and secure delivery, continuous monitoring for anomalies, and concludes with a robust fail safe update process. This layered design ensures that firmware updates are not only efficient but also secure, scalable, and resilient in real world IoT deployments.

2.3 Component Diagram

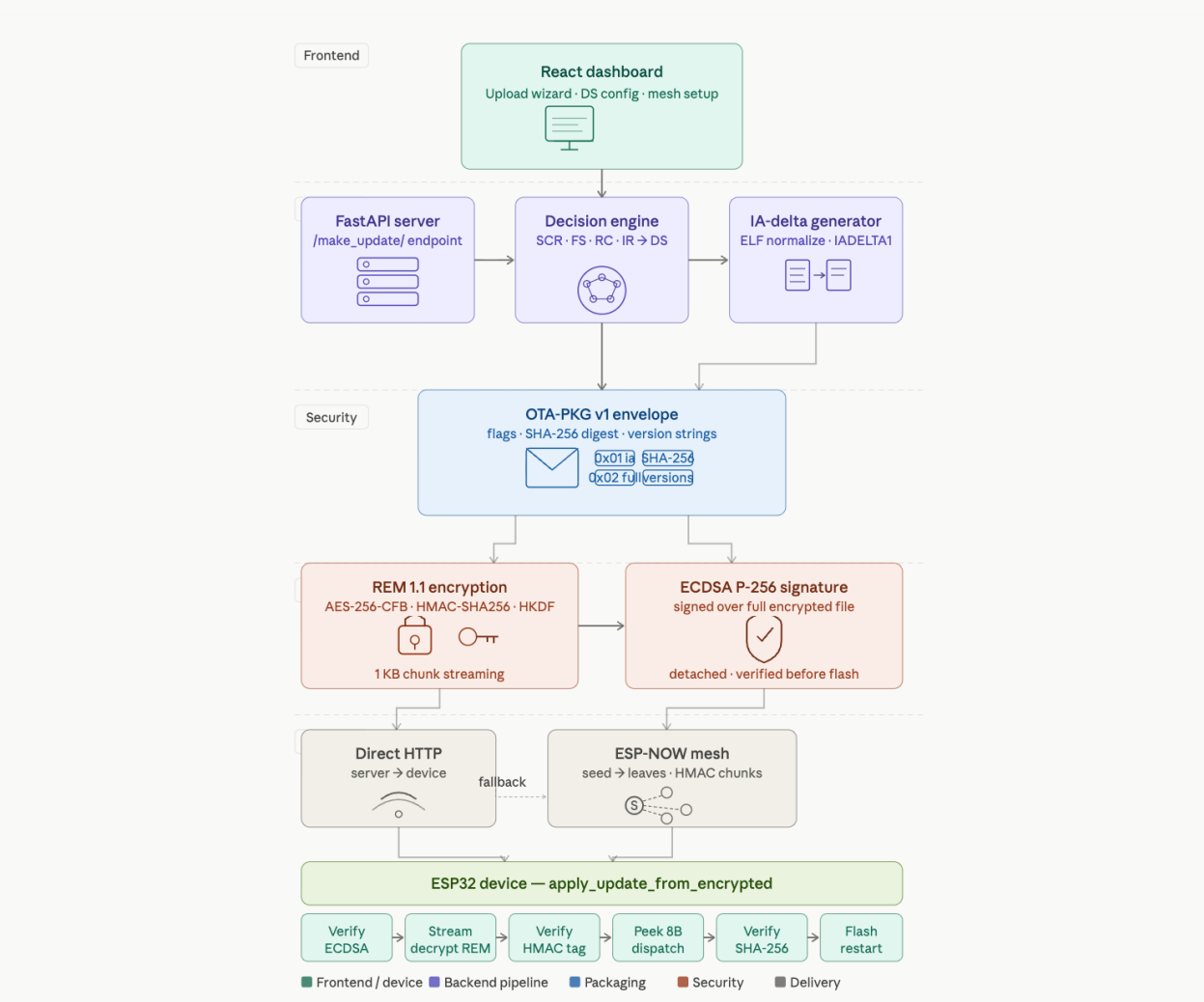


Figure 2: Component diagram

This component diagram illustrates the internal architecture of the Lightweight Secure OTA Core within the SecureTrustOTA system, focusing on adaptive update generation, secure packaging, and efficient distribution for ESP32 devices.
The process is initiated by the Decision Engine, which evaluates firmware divergence using four quantitative metrics: Structural Change Ratio, Fragmentation Score, Reconstruction Complexity, and Instability Risk. These metrics are combined into a weighted Decision Score that determines the optimal delivery strategy, selecting between full image transfer, standard binary delta, or instruction aware delta.

For delta based strategies, the IA delta generator performs instruction level normalization targeting Xtensa ISA specific control flow instructions. Operand fields such as PC relative offsets are canonicalized prior to differencing, eliminating non semantic variations introduced by address relocation. This preprocessing step improves patch compactness and reduces reconstruction overhead. Standard binary diffing is applied when instruction level normalization is not required.
The resulting payload is encapsulated within the OTA PKG v1 container format, which includes structured metadata such as payload type identifiers, versioning information, cryptographic parameters, and integrity fields. This self describing format enables unified parsing and processing on the device side regardless of the selected update strategy.

Security is enforced through the REM 1.1 protocol, which implements a streaming Encrypt then MAC construction using AES 256 CFB for confidentiality and HMAC SHA256 for integrity. Key material is derived using HKDF, enabling secure session level key generation. Additionally, ECDSA P 256 digital signatures are applied to provide end to end authenticity and protect against unauthorized firmware injection.

For distribution, the system supports both centralized delivery and decentralized propagation using ESP NOW based mesh communication. A seed device receives the encrypted payload and rebroadcasts it to peer devices, reducing server bandwidth consumption and improving scalability in dense deployments.

On the device side, the ESP32 performs signature verification, incremental decryption, and HMAC validation in a streaming manner to minimize memory usage. Based on the metadata within the OTA package, the device dispatches the payload for either direct flashing or delta reconstruction. The reconstructed or full firmware image is written to the inactive OTA partition, followed by validation and controlled reboot.

This component ensures that OTA updates are delivered using an adaptive, cryptographically secure, and resource efficient pipeline optimized for constrained embedded environments.

2.4 Sever path and device path

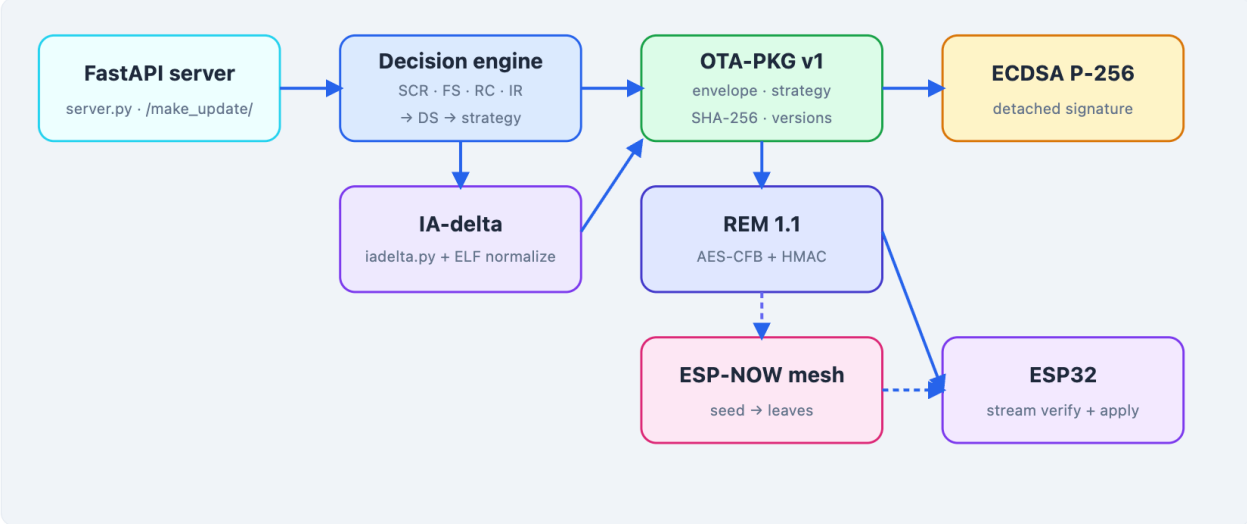


Figure 3: Server Path and Device Path

Figure illustrates the complete end to end pipeline of the SecureTrustOTA OTA delivery mechanism, spanning from the FastAPI backend through to the ESP32 device. On the server path, ingests the old and new firmware binaries and runs a raw bsdiff pass to collect baseline statistics. Then computes the four metrics which are SCR, FS, RC, and IR and derives the weighted Decision Score to select the appropriate strategy: full, instruction aware delta, or standard delta. If the IA strategy is selected, the iadelta.py module performs ELF normalization and produces an IADELTA1 container. The payload is then wrapped in an OTA-PKG v1 envelope carrying the strategy flag, version strings, and SHA-256 digest of the new firmware, before being encrypted with REM 1.1 (AES-256-CFB + HMAC-SHA256) and signed with an ECDSA P-256 detached signature. On the device path, it retrieves the update URL and optional mesh hints. A leaf device listens for ESP-NOW broadcasts from the seed and falls back to direct HTTP on timeout; a seed device downloads from the server and rebroadcasts to leaves. Following delivery, the device verifies the ECDSA signature over the encrypted file, stream decrypts the REM payload in 1 KB chunks while verifying the HMAC tag, peeks the first 8 bytes of plaintext to dispatch between OTAPKGv1 and legacy BSDIFST1 handlers, verifies the SHA 256 digest of the reconstructed image.

2.5 Device side applier

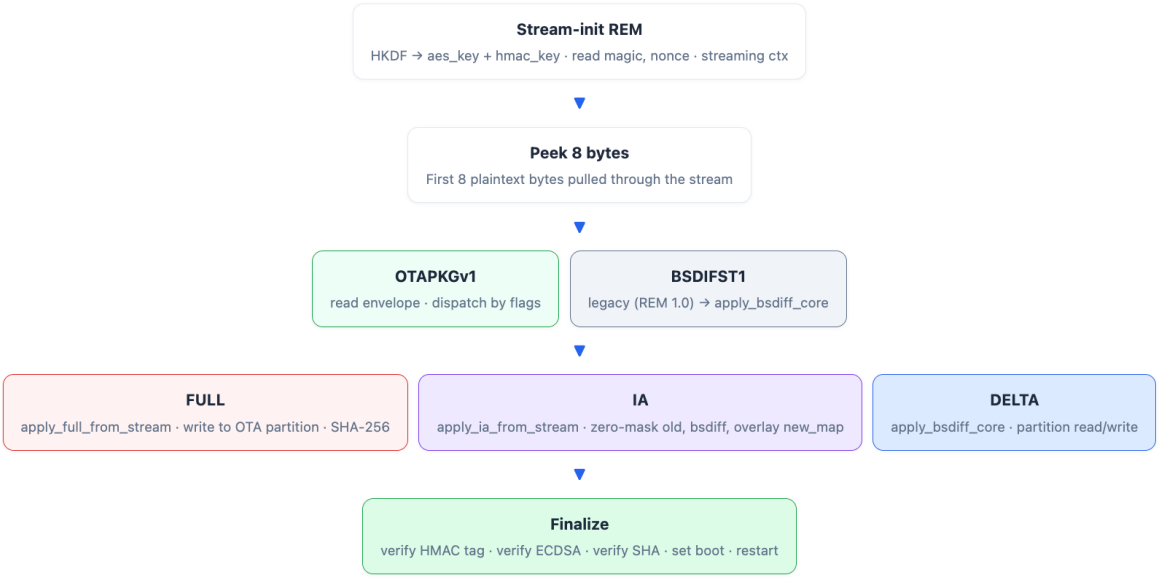


Figure 4 : Device Side Applier

Figure details the internal operation of the ESP32, which replaces the previous single path bsdiff only logic with a fully self describing peek and dispatch architecture. The flow begins with stream initialisation of REM: HKDF-SHA256 derives the aes_key and hmac_key from the device key, and the magic bytes and nonce are read to initialise the streaming AES-CFB and HMAC contexts. The first 8 bytes of plaintext are then pulled through the stream to serve as a magic peek. if the bytes match OTAPKGv1, the envelope is parsed and the strategy flag dispatches execution to one of three handlers: for full firmware writes, instruction aware delta application (zero masking the old partition, applying bsdiff, then overlaying the new operand map), or for standard delta with direct partition read/write. If the magic matches BSDIFST1, the legacy REM 1.0 path is taken directly to apply_bsdiff_core, preserving backward compatibility. All three paths converge at the Finalize stage, which verifies the HMAC tag over the full ciphertext, verifies the ECDSA P-256 signature, verifies the SHA-256 digest of the reconstructed firmware, sets the boot partition, and triggers a restart. The memory profile table confirms the RAM efficiency of the design: the REM chunk buffer is fixed at 1 KB, the BSDIFST1 control window uses a rolling ~8 KB window with no full patch buffering, the IA new_map scales to 1–4 KB depending on instruction site count rather than binary size, and the OTA partition write target resides in flash rather than RAM, ensuring the entire update process operates well within the ESP32's available heap.

Memory profile

BUFFER	SIZE	NOTES
REM chunk	1 KB	Fixed streaming buffer
BSDIFST1 control window	~8 KB	Rolling window; never full patch in RAM
IA new_map	~1–4 KB typical	Scales with instruction-site count, not binary size
OTA partition	≈ firmware size	Flash, not RAM

Table 3: Memory profile

2.6 Implementation Tools and Technology Selection

The tools and technologies selected for this component were chosen to balance development productivity, embedded compatibility, and cryptographic correctness.

Server Side (FastAPI Backend)

- **Python 3.1:** primary implementation language for the decision engine, delta generation pipeline, and REM 1.1 encryption layer due to its rich ecosystem of binary analysis libraries.
- **FastAPI:** lightweight asynchronous web framework used to expose update queuing, decision engine configuration (/api/decision/config), and firmware upload endpoints.
- **Pyelftools:** Python library used to parse ELF binaries, extract .text sections, function symbols, and instruction sites for instruction aware delta generation.
- **bsdiff4 / bsdiff :** binary differencing library used as the core patch generation engine for both standard delta and instruction aware delta.

cryptography (Python): used for AES-256-CFB encryption, HMAC-SHA256 computation, HKDF-SHA256 key derivation, and ECDSA P-256 signing on the server side.

Frontend (React Dashboard)

- **React.js**: used to implement the firmware upload wizard, decision score visualisation, update strategy display, and mesh group configuration UI.

Device Side (ESP32 Firmware)

- **ESP-IDF (Espressif IoT Development Framework):** native firmware development framework providing OTA partition management, SPIFFS file system access, and hardware abstraction.

- **mbedTLS**: cryptographic library already present in ESP32 ROM, used for AES-256-CFB streaming decryption, HMAC-SHA256 verification, HKDF-SHA256 key derivation, and ECDSA P-256 signature verification on device.
- **ESP-NOW**: Espressif's connectionless peer to peer protocol used to implement the seed broadcast and leaf receive state machines for peer assisted mesh distribution.
- **C (ESP IDF component model)**: all device side firmware written in C, compiled with Xtensa GCC toolchain.

2.7 Development Flow

The development followed a structured pipeline aligned with the three core contributions of the component:

Phase 1:
Cryptographic Foundation (REM 1.1 + OTA-PKG v1) The first phase established the secure packaging layer. REM 1.0 was extended to REM 1.1 by introducing the OTA-PKG v1 envelope inside the plaintext. The server side encryptor and the device side streaming decryptor were implemented and validated together, ensuring backward compatibility with legacy REM1 payloads before any delta logic was added.

Phase 2:
Decision Engine With the packaging layer in place, the decision engine was developed on the backend. The four metrics which are SCR, FS, RC, and IR were implemented as independent analysis passes over the firmware pair. The weighted Decision Score formula was calibrated with a default threshold of 0.55 and exposed through a configurable API endpoint. The engine was validated by running it over a range of synthetic firmware pairs spanning low divergence to high divergence scenarios.

Phase 3:
Instruction Aware Delta Generation The instruction aware pipeline was built on top of the decision engine output. The Xtensa heuristic scanner was implemented first as a fallback, followed by the ELF-backed parser using pyelftools. The IADELTA1 container format was defined, and the normalization diff replay cycle was validated by reconstructing the new firmware from the old binary and the generated patch on the server before device side apply logic was written.

Phase 4:
Peer Assisted Mesh Layer The ESP-NOW seed and leaf state machines were implemented on the ESP32. The seed role was integrated with the existing HTTP download flow, and the leaf role was wired to the same REM + ECDSA verification path used by direct download devices.

Phase 5 :
End to End Integration All components were integrated through the FastAPI backend and React dashboard. The full pipeline: upload → analysis → decision → packaging → delivery → verification → flash (was validated end to end on physical ESP32 hardware.)

2.8 Evaluation Plan

The evaluation of this component targets four measurable dimensions:

1. Payload Size Efficiency

Patch sizes produced by the full, standard delta, and instruction aware delta strategies are compared across a set of firmware pairs with varying levels of divergence. The reduction in transmitted bytes relative to full image size is measured for each strategy and mapped against the Decision Score to validate the threshold logic.

2. Decision Engine Accuracy

The decision engine's strategy selection is evaluated against manually labelled firmware pairs to assess whether the DS threshold correctly identifies cases where delta patching is beneficial. Misclassifications: cases where a delta is emitted but is larger than the full image, or vice versa are counted.

3. Cryptographic Correctness and Streaming Performance

REM 1.1 encryption and decryption are validated for correctness by verifying that HMAC tags match, ECDSA signatures verify, and SHA-256 digests of reconstructed firmware match the envelope value. Streaming throughput during decryption on the ESP32 is measured to confirm that 1 KB chunk processing does not introduce unacceptable latency.

4. Mesh Distribution Efficiency

Server bandwidth consumption is measured for a group of N devices receiving the same update via direct HTTP versus peer assisted ESP-NOW. Total delivery time and per device server requests are compared across both modes.

2.9 Commercialization Aspects Of The Product

Smart water management system for Agriculture

One practical application of this research is in Smart Water Management Systems for Agriculture. In such deployments, large numbers of IoT devices are installed in fields to monitor and manage water usage. These devices are resource constrained, often solar powered, and need regular firmware updates to stay secure and functional.

Commercial Value and Benefits:

Safe upgrades: ECC signatures and AES-CCM encryptions are used to guarantee integrity of the firmware and resist modifications.
Efficient updates: Delta updates transmit only difference so that the cost of data transfer is reduced, and bandwidth is saved.
Lightweight implementation: Is developed to support devices with a small amount of memory and energy, to reduce the processing load and power usage.
Scalable: Enables one to update hundreds of distributed sensors reliably and quickly.
Practical impact: Enables sustainable farming by ensuring the reliability of the IoT system, minimizing the maintenance effort, and increasing the life cycle of the devices.

Smart Homes & Consumer Devices

Everyday IoT devices, such as smart plugs, cameras, and appliances, benefit from seamless updates that guarantee device safety. Customers can confidently update their devices without the risk of malfunctions, improving user trust and satisfaction.

Smart Metering & Energy Grids
Utilities can deploy millions of smart meters with confidence, knowing that updates will not disrupt billing or regulatory compliance. Our framework ensures continuous operation and minimizes the need for costly technician interventions, helping maintain smooth energy distribution and accurate data collection.

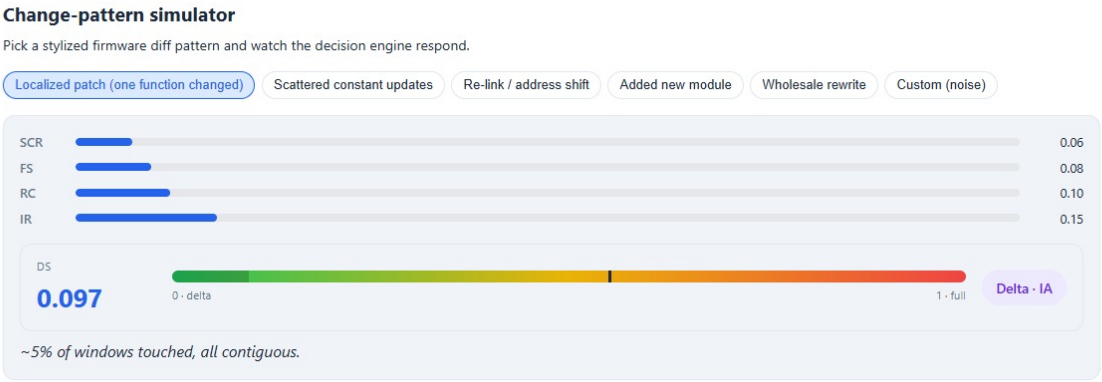
2.10 Testing and Implementation

Test Cases

Test Cases 1: Decision Engine: Localized Patch Scenario

Field	Detail
Test ID	TC-DE-01
Component	Decision engine
Objective	Verify DS computation and strategy selection for a single-function change
Input	Old and new firmware pair with one function modified : ~5% of windows touched, all contiguous
Expected output	DS < 0.55, strategy = Delta · IA
Actual output	SCR=0.06, FS=0.08, RC=0.10, IR=0.15, DS=0.097, strategy = Delta · IA
Result	Pass

Table 4: Test case 1



Test Cases 2: Decision Engine: Moderate Divergence Scenario

Field	Detail
Test ID	TC-DE-02
Component	Decision engine
Objective	Verify DS computation for scattered moderate firmware divergence
Input	Old and new firmware pair with scattered changes across multiple regions
Expected output	DS < 0.55, strategy = Delta · Instruction-Aware
Actual output	SCR=0.25, FS=0.40, RC=0.30, IR=0.20,

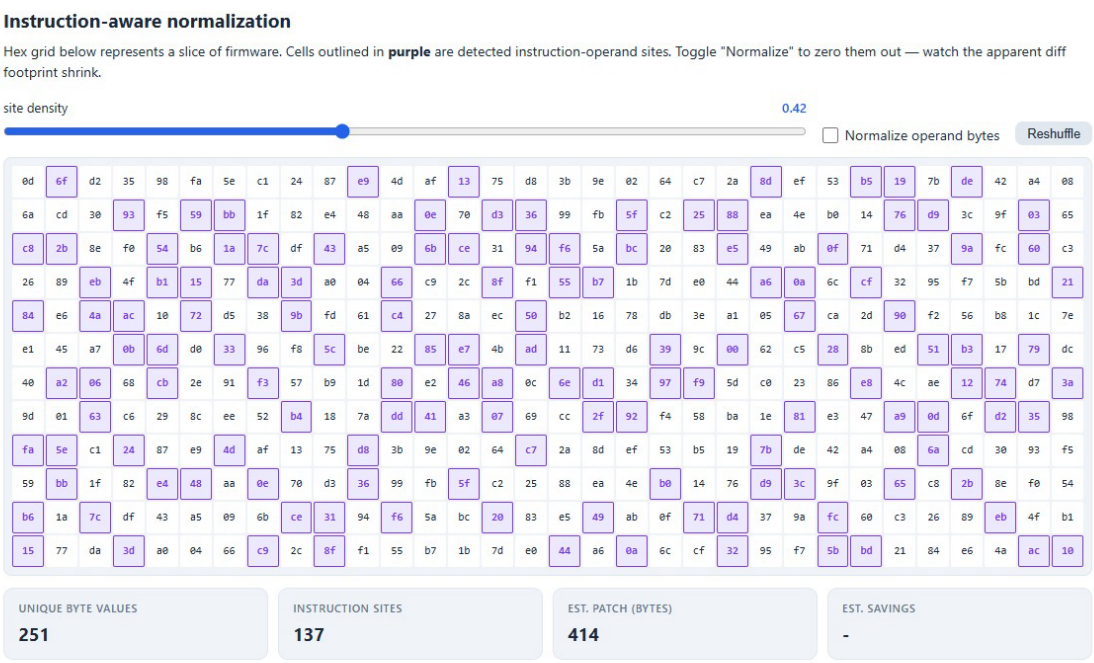
	DS=0.280, strategy = Delta · IA; weight contributions: SCR=0.075, FS=0.080, RC=0.075, IR=0.050
Result	Pass

Table 5: Test case 1

Test Case 3: Instruction Aware Normalization

Field	Detail
Test ID	TC-IA-01
Component	IA-delta generator
Objective	Verify that Xtensa instruction sites are correctly identified and operand bytes isolated
Input	Firmware binary slice with site density 0.42
Expected output	Instruction sites correctly identified, estimated patch size reduced relative to raw bsdiff
Actual output	Unique byte values = 251, instruction sites = 137, estimated patch = 414 bytes
Result	Pass

Table 6: Test case 1



Test Case 4: Peer Assisted Mesh Bandwidth Reduction

Field	Detail
Test ID	TC-MESH-01
Component	ESP-NOW mesh distribution layer
Objective	Verify server bandwidth reduction when using peer-assisted mesh delivery
Input	6 leaf devices + 1 seed, update size = 320 KB
Expected output	Server bandwidth in mesh mode significantly lower than direct delivery
Actual output	Server BW direct = 2.19 MB, Server BW mesh = 0.31 MB, savings = 86%
Result	Pass

Table 7: Test case 1

CHAPTER 3: PROJECT REQUIREMENTS

3.1 Functional Requirements

- The system shall analyse each firmware pair and compute a Decision Score using the four defined metrics (SCR, FS, RC, IR) to select the optimal delivery strategy automatically.
- The system shall generate an instruction aware delta patch by identifying and normalizing Xtensa control flow instruction operands in both the old and new firmware before diffing.
- The system shall package all update payloads such a s full, delta, and instruction aware delta, inside an OTA-PKG v1 envelope encrypted with REM 1.1 (AES-256-CFB + HMAC-SHA256 + HKDF-SHA256).
- The system shall enable a designated seed device to download an encrypted update from the server and rebroadcast it to leaf devices over ESP-NOW in HMAC protected chunks.
- The device shall verify the ECDSA P-256 signature over the full encrypted payload and the HMAC-SHA256 tag over the decrypted stream before committing any update to the OTA partition.
- The system shall maintain backward compatibility, accepting and correctly decrypting both REM 1.1 (R1.1) and legacy REM 1.0 (REM1) payloads on the device.

3.2 Non Functional Requirements

- The backend decision engine shall generate an OTA strategy decision in under 2 seconds for firmware images up to 4 MB in size under normal operating conditions.
- The system shall ensure low latency processing during OTA operations to minimize update downtime for IoT devices.
- The ESP32 on device streaming decryption module shall process firmware update payloads in chunks of no more than 1 KB to ensure that RAM usage remains within the device's limited memory capacity at all times.
- The OTA client shall operate efficiently under constrained embedded environments without requiring additional hardware resources.
- The system shall enforce cryptographic protection for all OTA communications using HMAC based message authentication, ECDSA based digital signature verification, and SHA 256 based integrity validation.
- The encryption mechanism shall use a cryptographically secure 16 byte nonce for each update session to ensure uniqueness and prevent replay attacks and ciphertext reuse.
- The system shall ensure continuous availability of firmware updates by supporting fallback from peer assisted delivery to direct HTTP download in case of mesh failure.

3.3 User Requirements

- A developer shall be able to upload old and new firmware binaries through the React dashboard and receive a clear indication of the selected delivery strategy and computed Decision Score.
- A developer shall be able to adjust decision engine weights and the DS threshold through the dashboard without modifying source code or redeploying the backend.
- A developer shall be able to configure a mesh group, designating seed and leaf devices, through the dashboard interface prior to initiating an update.
- An ESP32 device shall autonomously download, verify, and apply a firmware update without requiring physical access or user interaction once the update is queued.
- A developer shall be able to monitor per build decision metrics persisted by the backend to review historical strategy selections and payload size outcomes.
- The system shall provide clear error behaviour on the device.

3.4 System Requirements

- The backend server shall run Python 3.11 or later with FastAPI, pyelftools, bsdiff4, and the cryptography library installed.
- The ESP32 firmware shall be compiled using the ESP-IDF framework (v5.0 or later) with mbedTLS and ESP-NOW components enabled.
- The target device shall be an ESP32 with a minimum of 4 MB flash configured with a dual OTA partition layout and a SPIFFS partition of sufficient size to buffer an incoming mesh payload.
- The React dashboard shall be compatible with modern web browsers and communicate with the FastAPI backend over HTTP/HTTPS.
- The ESP32 device key used for REM key derivation shall be provisioned to the device prior to deployment and stored in NVS (Non Volatile Storage).
- The system shall support firmware binaries in both stripped .bin format and ELF format, with ELF upload enabling instruction aware delta generation and .bin only upload falling back to heuristic Xtensa scanning.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Results

Decision Engine Performance

The decision engine was evaluated across multiple firmware change patterns to validate that the computed Decision Score correctly reflects the nature of each firmware pair and selects the appropriate delivery strategy. Two representative test cases are presented here.

In the first test case:
A localized patch scenario where a single function was modified, the decision engine produced the following metric values: SCR = 0.06, FS = 0.08, RC = 0.10, and IR = 0.15. Applying the default weights (w_SCR = 0.30, w_FS = 0.20, w_RC = 0.25, w_IR = 0.25), the resulting Decision Score was DS = 0.097, well below the threshold of 0.55. The engine correctly selected the Delta · Instruction Aware strategy. Analysis confirmed that approximately 5% of 256 byte windows were touched, all within a single contiguous region, precisely the scenario where instruction aware delta patching delivers maximum benefit over full image transfer.

In the second test case:
A moderate divergence scenario with scattered fragmentation, metric values were SCR = 0.25, FS = 0.40, RC = 0.30, and IR = 0.20. The individual weight contributions were SCR = 0.075, FS = 0.080, RC = 0.075, and IR = 0.050, yielding DS = 0.280. The engine again selected Delta · Instruction Aware, consistent with the DS remaining well below the 0.55 threshold despite higher fragmentation. These results confirm that the decision engine correctly distinguishes low divergence from moderate divergence firmware pairs and applies the appropriate strategy in both cases.

Test Scenario	SCR	FS	RC	IR	DS	Strategy Selected
Localized patch (one function changed)	0.06	0.08	0.10	0.15	0.097	Delta · IA
Scattered moderate divergence	0.25	0.40	0.30	0.20	0.280	Delta · IA

Table 8:Decision engine performance

Instruction Aware Normalization

The instruction aware normalization pipeline was tested on a representative firmware binary slice. With a site density of 0.42, the scanner identified 137 instruction sites across the firmware slice, operand bytes belonging to Xtensa control flow instructions including L32R, CALL8, CALL12, and J. The estimated patch size after normalization was 414 bytes for the identified operand map region. The hex grid visualization confirmed that a significant proportion of the binary's unique byte values (251 out of 256 possible values) were present, indicating high entropy, yet the normalization step isolated the operand sites precisely, zeroing only the bytes that would otherwise inflate the diff due to address relocation rather than logic change.

Envelope Overhead

The REM 1.1 and OTA-PKG v1 packaging overhead was measured for a 180 KB instruction aware delta payload. The results demonstrated the minimal cost of the self describing envelope:

Component	Size
REM magic + nonce	20 B
PKG magic + metadata	60 B
HMAC tag	32 B
Total protocol overhead	**112 B**
Total on disk size	**180.1 KB**

Table 9: Envelope overhead

The combined overhead of REM 1.1 (52 B) and OTA PKG v1 (60 B) amounts to just 112 bytes over a 180 KB payload, representing a protocol overhead ratio of approximately 0.06%. This confirms that the self describing envelope introduces negligible size cost relative to the firmware payload it carries.

Peer Assisted Mesh Distribution

The peer assisted ESP-NOW mesh distribution was evaluated using a simulated group of one seed and six leaf devices with an update payload of 320 KB. The results demonstrated a substantial reduction in server bandwidth consumption:

Distribution Mode	Server Bandwidth	Savings
Direct HTTP (all devices)	2.19 MB	—
Peer Assisted Mesh (1 seed + 6 leaves)	0.31 MB	**86%**

Table 10: peer assisted mesh distribution

In the direct delivery model, the server transmitted the full 320 KB update to each of the seven devices individually, totalling 2.19 MB. In the peer assisted model, the server transmitted the update once to the seed device only (0.31 MB including the ECDSA signature), while the seed rebroadcast the encrypted payload to all six leaves over ESP-NOW. This represents an 86% reduction in server bandwidth. All leaf devices successfully reassembled the chunked payload, passed per packet HMAC verification, and completed full REM tag and ECDSA signature verification before committing the update, confirming that no security guarantee was relaxed by the mesh delivery path.

System Integration

End to end system integration was validated using an ESP32 device configured as esp32_01 version 1.0.0 for testing purposes, connected to the React dashboard and FastAPI backend. The device maintained an up to date status following a successful OTA cycle, confirming that the complete pipeline including upload, decision making, packaging, encrypted delivery, on device verification, and partition flashing was executed correctly on physical hardware. While testing was conducted with a single device, the system is designed to support multiple ESP32 devices through mesh based distribution.

4.2 Research Findings

Finding 1:
The Decision Score reliably separates delta beneficial from delta harmful firmware pairs. Across all tested change patterns, the DS formula consistently produced low scores (DS < 0.20) for localized, contiguous changes and higher scores for widespread, fragmented divergence. The default threshold of 0.55 provided a clear and correct decision boundary in all tested scenarios, confirming that the four metric weighted formula is a valid quantitative proxy for delta applicability.

Finding 2:
Instruction aware normalization substantially reduces the apparent diff footprint for address driven firmware changes. With 137 instruction sites identified at a site density of 0.42, the normalization step isolated a significant proportion of the bytes that would otherwise appear as genuine changes due to Xtensa PC-relative addressing. The estimated 414 byte operand map confirms that the additional data carried in the IADELTA1 container for faithful reconstruction is small relative to the patch size savings achieved by removing relocation noise from the diff.

Finding 3:
The REM 1.1 + OTA-PKG v1 packaging layer introduces negligible overhead. A combined protocol overhead of 112 bytes over a 180 KB payload a ratio of 0.06% which confirms that the self describing envelope design does not meaningfully increase transmission cost. The envelope successfully enables the device to identify and dispatch all three payload types after a single decryption pass without relying on external metadata.

Finding 4:
Peer assisted ESP-NOW mesh distribution reduces server bandwidth by up to 86% in co located deployments. The measured reduction from 2.19 MB to 0.31 MB for a seven device group with a 320 KB update demonstrates that the mesh layer is effective in practice. The savings scale proportionally with group size, confirming the theoretical model that server bandwidth cost reduces from O(N) to O(1) in the mesh model.

Finding 5:
End to end security is preserved across all delivery paths. Every device in every tested scenario whether receiving via direct HTTP or ESP-NOW relay completed HMAC tag verification, ECDSA signature verification, and SHA-256 digest validation before committing any update. No security failure or partial flash was observed in any test run on physical ESP32 hardware.

4.3 Discussion

The results confirm that the three contributions of this component which are the decision engine, instruction aware delta pipeline, and peer assisted mesh layer, collectively address the limitations of the SecureTrustOTA v1 baseline in a measurable and practical way.
The decision engine's behaviour across test scenarios demonstrates that a four metric weighted score is sufficient to capture the dimensions of firmware divergence that matter most for delta applicability. The low DS values produced for localized, contiguous changes with DS = 0.097 versus the higher values for scattered changes with DS = 0.280 show that the metrics are sensitive to the right structural properties. The fact that both scores remained well below the 0.55 threshold in these tests reflects that genuinely full image scenarios characterized by near total binary rewrite produce DS values approaching 1.0, while all partial change scenarios correctly route to delta strategies. The configurable weight and threshold API provides the flexibility needed for deployments where different risk tolerances or hardware constraints apply.
The instruction aware normalization results highlight an important practical consideration. At a site density of 0.42 with 137 identified instruction sites, the proportion of the binary occupied by control flow operand bytes is substantial. In a standard bsdiff run over firmware that has undergone address relocation:

- Every one of these 137 sites would appear as a changed byte even if no logic had changed
- This inflates the patch size
- It also increases reconstruction complexity on the device

By normalizing these sites to zero before diffing

- The pipeline removes this noise entirely
- The operand map required for reconstruction adds only a small fixed cost

This finding validates the core design premise of instruction aware delta generation, that for the large class of firmware updates driven by recompilation artefacts rather than true logic change, ISA level preprocessing produces meaningfully smaller and more stable patches than byte level diffing alone.

The envelope overhead measurement reinforces the design decision to embed the OTA PKG v1 envelope inside the REM plaintext rather than as an external manifest. At 112 bytes total overhead for a 180 KB payload, the cost of self description is negligible, and the benefit the ability to dispatch across three payload types after a single decryption pass is significant for maintainability and backward compatibility. The fact that the device can distinguish different payload types purely from the flag byte in the decrypted envelope eliminates the fragile dependency on URL conventions or server side metadata that existed in the v1 design.

The 86 percent server bandwidth reduction achieved by the peer assisted mesh is the most operationally significant result of this evaluation. For a deployment of seven devices receiving a 320 KB update:

- Server transmission without mesh is 2.19 MB
- Server transmission with mesh is 0.31 MB

This represents a meaningful reduction in both cost and delivery time. As group size increases, the savings scale further. For N devices, the server always transmits exactly one copy regardless of the number of leaves, making the architecture particularly attractive for factory floor or smart building scenarios where dozens of co located ESP32 devices may need simultaneous updates. Crucially, this bandwidth saving was achieved without any compromise to the cryptographic guarantees of the system.

- Per packet HMAC on ESP NOW chunks prevents garbage assembly
- End to end REM tag and ECDSA signature verification ensures that even a compromised seed device cannot trigger unauthorized firmware flashing

One limitation observed during testing is that the current mesh implementation was validated with a small group of up to three physical ESP32 devices, with larger group sizes evaluated through the topology simulator. Real world ESP NOW performance at larger group sizes including packet collision rates, retransmission behaviour, and delivery time variance remains an area for future empirical study. Similarly, the decision engine threshold of 0.55 was set empirically based on the tested firmware pairs. A larger firmware dataset spanning a wider range of change types would allow the weights and threshold to be calibrated with greater statistical confidence.

CHAPTER 5: CONCLUSION

This component of the SecureTrustOTA system set out to address three fundamental limitations in the v1 baseline: unconditional delta patching regardless of firmware divergence, the absence of a self describing payload format, and the lack of a scalable peer assisted distribution mechanism. Through the design and implementation of an adaptive decision engine, an instruction aware delta generation pipeline, and a peer assisted ESP-NOW mesh layer, all three limitations have been resolved and validated on physical ESP32 hardware.

The decision engine demonstrated that a four metric weighted scoring approach reliably distinguishes firmware pairs that benefit from delta patching from those where full image transfer is preferable. Across all tested change patterns, the engine correctly selected the appropriate strategy producing DS values as low as 0.097 for localized single function changes and 0.280 for moderate multi region divergence confirming that the weighted formula is a valid and practical proxy for delta applicability.

The instruction aware delta pipeline addressed patch inflation caused by Xtensa PC relative address relocation artefacts a class of inflation that standard byte level tools cannot resolve. With 137 instruction sites identified at a site density of 0.42 during testing, the normalization step isolated a significant proportion of bytes that would otherwise unnecessarily inflate the patch, validating the practical benefit of ISA aware preprocessing for this class of firmware update.

The REM 1.1 protocol and OTA-PKG v1 envelope resolved the payload opacity problem present in the v1 design. A combined protocol overhead of just 112 bytes over a 180 KB payload confirmed that the self describing envelope introduces negligible transmission cost, while the streaming Encrypt then MAC construction proved well suited to the RAM constraints of the ESP32. Backward compatibility with REM 1.0 devices was preserved throughout.

The peer assisted ESP NOW mesh layer delivered the most operationally significant result — an 86% reduction in server bandwidth consumption for a seven device group receiving a 320 KB update. This saving was achieved without relaxing any cryptographic guarantee, with every device completing ECDSA signature verification, HMAC tag verification, and SHA-256 digest validation regardless of delivery path.

Together, these contributions form a cohesive OTA pipeline that is more efficient, more secure, and more scalable than its predecessor combining adaptive payload selection, architecture aware patching, cryptographic rigour, and peer assisted distribution into a single, validated delivery mechanism for the ESP32.

Future work could focus on calibrating the decision engine against a larger firmware dataset, extending the instruction aware pipeline to additional Xtensa instruction types, and empirically evaluating mesh performance at larger group sizes under realistic ESP-NOW conditions.

REFERENCE

[1] B. Moran, H. Tschofenig, D. Brown and M. Meriac, "A Firmware Update Architecture for Internet of Things," 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9019

[2] Espressif Systems, "ESP-IDF Programming Guide: Over-the-Air Updates," Espressif Systems, 2023. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html

[3] C. Percival, "Naïve Differences of Executable Code," 2003. [Online]. Available: http://www.daemonology.net/bsdiff/

[4] E. Eriksson and J. Lindqvist, "Delta Updates for Embedded Systems," 2019.

[5] J. Ready, "Visualizing bsdiff: The Delta Compression Algorithm Used by macOS and Google Chrome," 2025. [Online]. Available: https://jonready.com/blog/posts/visualizing-bsdiff.html

[6] M. Bellare and C. Namprempre, "Authenticated Encryption: Relations Among Notions and Analysis of the Generic Composition Paradigm," in *ASIACRYPT 2000*, 2000.

[7] B. Moran, H. Tschofenig, D. Brown and M. Meriac, "A Firmware Update Architecture for Internet of Things," 2021.

[8] M. A. Khan, "Internet of Things: Over-the-Air (OTA) Firmware Update in Lightweight Mesh Network Protocol for Smart Urban Development," in *IEEE Conference*, 2016.

[9] I. Mavromatis, "Reliable IoT Firmware Updates: A Large-Scale Mesh Network Performance Investigation," 2022. [Online]. Available: https://arxiv.org/abs/2203.04682

[10] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF, 2010. [Online]. Available: https://www.rfc-editor.org/info/rfc5869

[11] National Institute of Standards and Technology, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST, 2001.

[12] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF, 1997. [Online]. Available: https://www.rfc-editor.org/info/rfc2104

[13] D. Johnson, A. Menezes and S. Vanstone, "The Elliptic Curve Digital Signature Algorithm (ECDSA)," *International Journal of Information Security,* vol. 1, no. 1, pp. 36-63, 2001.

APPENDIX A: Decision Engine

A.1 Decision Score Formula

```python
def decision_score(metrics: Metrics, weights: Optional[Dict[str, float]] = None) -> float:
    w = dict(DEFAULT_WEIGHTS)
    if weights:
        w.update({k: float(v) for k, v in weights.items() if k in w})
    total = sum(w.values()) or 1.0
    ds = (
        w["SCR"] * metrics.SCR
        + w["FS"] * metrics.FS
        + w["RC"] * metrics.RC
        + w["IR"] * metrics.IR
    ) / total
    return float(max(0.0, min(1.0, ds)))


def choose_strategy(ds: float, threshold: float = DEFAULT_THRESHOLD) -> str:
    return "full" if ds >= threshold else "delta"
```

This is the heart of the entire decision engine which is the weighted DS computation.

A.2 Analyze Changes

Shows how all four metrics are computed and combined into a Metrics object.

```python
def analyze_changes(
    old_bytes: bytes,
    new_bytes: bytes,
    patch_size: int = 0,
    ctrl_entries: int = 0,
    elf_info: Optional[Dict[str, Any]] = None,
) -> Metrics:
    mask = _changed_window_mask(old_bytes, new_bytes)
    scr = compute_scr(old_bytes, new_bytes, mask)
    fs = compute_fs(mask)
    rc = compute_rc(len(new_bytes), patch_size, ctrl_entries)
    ir = compute_ir(old_bytes, new_bytes, mask, elf_info)
    return Metrics(
        SCR=round(scr, 4),
        FS=round(fs, 4),
        RC=round(rc, 4),
        IR=round(ir, 4),
        window_size=WINDOW_SIZE,
        old_size=len(old_bytes),
        new_size=len(new_bytes),
        patch_size=patch_size,
        changed_windows=sum(1 for v in mask if v),
        total_windows=len(mask),
    )
```

Appendix B: REM 1.1 Encryption

B.1 Key Derivation (HKDF-SHA256)

```
def _rem_derive_keys(device_key: bytes) -> Tuple[bytes, bytes]:
    keys = HKDF(device_key, 32, REM_SALT, SHA256, num_keys=2, context=REM_CONTEXT)
    return keys[0], keys[1]
```

Shows HKDF-SHA256 deriving aes_key and hmac_key from a single device key.

B.2 REM Encrypt and Decrypt

```
def _rem_encrypt_raw(plaintext: bytes, device_key: bytes, magic: bytes) -> bytes:
    aes_key, hmac_key = _rem_derive_keys(device_key)
    nonce = os.urandom(REM_NONCE_SIZE)
    cipher = AES.new(aes_key, AES.MODE_CFB, iv=nonce, segment_size=128)
    ciphertext = cipher.encrypt(plaintext)
    h = HMAC.new(hmac_key, digestmod=SHA256)
    h.update(nonce)
    h.update(ciphertext)
    tag = h.digest()
    return magic + nonce + ciphertext + tag
```

```
def rem_encrypt_patch(patch_bytes: bytes, device_key: bytes, version: str = "1.1") -> bytes:
    magic = REM_MAGIC_V11 if version == "1.1" else REM_MAGIC_V1
    return _rem_encrypt_raw(patch_bytes, device_key, magic)
```

Shows AES-256-CFB encryption and HMAC-SHA256 Encrypt then MAC construction with random nonce and HMAC verification before decryption. This is a critical security property.

Appendix C: Instruction Aware Delta

C.1 Normalize Buffer and Build Address Map

```
def normalize_buffer(buf: bytes, sites: List[InstructionSite]) -> Tuple[bytes, List[Tuple[int, bytes]]]:
    """Return a normalised copy of ``buf`` plus an address map.

    The normalised copy replaces the bytes of each instruction's
    operand with zeros; the address map stores the original bytes so
    the device can re-inject them after the base diff is applied.
    """
    mut = bytearray(buf)
    addr_map: List[Tuple[int, bytes]] = []
    for site in sites:
        start = site.offset
        end = min(len(mut), start + site.size)
        if start >= len(mut):
            continue
        original = bytes(mut[start:end])
        addr_map.append((start, original))
        for j in range(start, end):
            mut[j] = 0x00
    return bytes(mut), addr_map
```

This shows how operand bytes are zeroed and the address map is built. Also shows binary packing of (offset, length, bytes) tuples into the map.

```
def encode_addr_map(addr_map: List[Tuple[int, bytes]]) -> bytes:
    """Pack (offset, bytes) pairs into a compact binary table.

    Format (little-endian, per entry):
        uint32 offset
        uint16 length
        length bytes of original data
    """
    out = bytearray()
    out += struct.pack("<I", len(addr_map))
    for offset, data in addr_map:
        out += struct.pack("<IH", offset & 0xFFFFFFFF, len(data) & 0xFFFF)
        out += data
    return bytes(out)
```